Algorithmique Fondamentale et Structures de Contrôle

Thème principal : Introduction à l'algorithmique impérative, typage, structures conditionnelles et boucles

Synthèse exécutive : Ce cours pose les fondations indispensables de l'algorithmique impérative en BUT 1 Informatique. Il formalise le passage d'un problème métier à sa résolution automatisée par la machine, en détaillant la déclaration typée des variables, l'évaluation rigoureuse des expressions booléennes (avec la notion critique d'évaluation court-circuit) et la maîtrise des structures itératives bornées et non bornées accompagnées de leurs invariants et variants de boucle.

Fondements Algorithmiques, Organisation Mémoire et Typage des Données

L'ordinateur exécute aveuglément des instructions codées en binaire. L'algorithmique constitue l'étape amont de conception formelle, totalement indépendante de tout langage de programmation spécifique. Avant toute manipulation, la machine doit réserver un espace d'adressage en mémoire centrale (RAM). Le type d'une donnée définit précisément le nombre d'octets alloués à la case mémoire ainsi que l'ensemble des opérations arithmétiques ou logiques licites. L'affectation modifie l'état de la mémoire de manière destructrice en remplaçant l'ancienne valeur par la nouvelle.

- Algorithme : suite finie, déterministe et non ambiguë d'instructions élémentaires opérant sur des données pour produire un résultat déterminé.
- Variable : emplacement mémoire nommé doté d'un identificateur, d'une adresse physique en RAM, d'un type et d'une valeur courante.
- Types primitifs : Entier (Integer), Réel (Float), Booléen (Boolean) et Chaîne de caractères (String).
- Affectation destructrice : opération (<- ou =) qui évalue l'expression à droite et écrase irrésistiblement le contenu de la case mémoire cible à gauche.

Cas concret enseignant : *La métaphore de la boîte aux lettres normalisée : une variable agit comme une boîte aux lettres portant une étiquette (l'identificateur) et dotée d'une fente aux dimensions fixes (le type). On ne peut pas introduire un carton volumineux dans une fente de carte postale. Le nouveau courrier détruit et remplace instantanément l'ancien.*

Prise de Décision : Algèbre Booléenne et Évaluation Court-Circuit

Les structures de contrôle conditionnelles s'appuient sur des prédicats combinant des opérateurs relationnels et logiques. L'évaluation court-circuit (par exemple dans 'A ET B' où 'A' est faux) évite d'évaluer inutilement la suite et prévient les erreurs d'exécution fatales comme les pointeurs nuls ou les dépassements de tableaux.

- Structure alternative : blocs 'Si (condition) Alors ... SinonSi ... Sinon ... FinSi' pour dévier le flux séquentiel d'instructions.
- Prédicat booléen : expression logique dont le résultat est strictement binaire (Vrai ou Faux).
- Évaluation court-circuit : stratégie d'optimisation où l'évaluation des conditions s'arrête dès que le résultat final est globalement certain.

Cas concret enseignant : *Le portillon d'accès au réseau de transport d'Anglet/Bayonne : pour valider le passage, la condition est '(Titre_Valide ET Solde_Suffisant)'. Si le titre est expiré, le tourniquet bloque immédiatement l'usager sans interroger le serveur bancaire central, économisant du réseau et évitant un temps d'attente.*

Structures Répétitives : Boucles Déterminées et Indéterminées

L'automatisation repose sur la répétition contrôlée. Si le nombre d'itérations est connu, on utilise une boucle 'Pour'. Sinon, on utilise une boucle 'Tant Que' en veillant impérativement à définir un variant de boucle pour éviter l'infinisation et assurer la terminaison de l'algorithme.

• Boucle bornée (Pour / For) : structure déterministe où le nombre total d'itérations est connu à l'avance.

• Boucle non bornée (Tant Que / While) : répétition assujettie à une condition booléenne, indispensable lorsque la fin dépend d'un événement externe.

• Variant de boucle : quantité entière strictement positive qui décroît à chaque itération pour garantir la terminaison.

Cas concret enseignant : *Le distributeur automatique de billets : l'automate ne sait pas d'avance combien de billets délivrer. Il applique un 'Tant Que (Somme_Restante > 0)', en soustrayant à chaque itération la valeur du billet distribué. La variable 'Somme_Restante' joue le rôle du variant de boucle.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours employer une boucle 'Tant Que' dans nos programmes, sachant qu'elle peut techniquement simuler n'importe quelle boucle 'Pour' ?
Réponse du professeur : Parce que la boucle 'Pour' est plus lisible et sécurisée lorsque le nombre d'itérations est connu à l'avance, évitant d'oublier l'incrémentation de la variable de contrôle qui mènerait à une boucle infinie.

Question étudiant : Quelle est la différence fondamentale entre l'opérateur 'OU' logique standard et l'opérateur 'OU EXCLUSIF' (XOR) ?
Réponse du professeur : Le 'OU' standard est vrai si au moins l'une des conditions est vraie (y compris les deux). Le 'OU EXCLUSIF' est vrai si et seulement si une seule des conditions est vraie, mais pas les deux en même temps.

■ Exercice & Démonstration d'Application

Énoncé : Concevoir un algorithme robuste qui demande à l'utilisateur de saisir une suite de notes réelles strictement positives et calcule leur moyenne arithmétique. La saisie s'interrompt dès que l'utilisateur entre la valeur sentinelle -1. Si l'utilisateur saisit des notes négatives aberrantes (différentes de -1), elles doivent être ignorées avec un message d'alerte. Si aucune note valide n'a été saisie, l'algorithme doit avertir l'utilisateur sans provoquer d'erreur de division par zéro.

Démarche de résolution :
1. Déclaration et initialisation des accumulateurs : somme <- 0.0 (Réel) et nb_notes <- 0 (Entier). 2. Lecture anticipée (amorce de la boucle sentinelle) : Afficher 'Note (-1 fin) :', lire 'note'. 3. Boucle principale : Tant Que (note != -1.0) Faire. 4. Vérification interne : Si (note > 0.0) Alors somme <- somme + note, nb_notes <- nb_notes + 1 Sinon Afficher 'Avertissement : Note invalide'. FinSi. 5. Réitération de la lecture en fin de boucle : lire 'note'. 6. FinTantQue. 7. Traitement post-boucle : Si (nb_notes > 0) Alors moyenne <- somme / nb_notes et afficher la moyenne, Sinon afficher 'Aucune note valide'. FinSi.

Résultat : L'algorithme traite correctement la condition d'arrêt sentinelle sans l'inclure dans le calcul, filtre les valeurs aberrantes négatives et sécurise le bloc arithmétique final contre le crash de division par zéro si -1 est saisi dès la première entrée.

■ Auto-Évaluation Flash d'Examen

Q : Que vaut l'expression booléenne : NON((5 > 2) ET (3 == 4)) ?

R : Vrai (car 5 > 2 est Vrai, mais 3 == 4 est Faux, donc le ET donne Faux, et la négation NON(Faux) donne Vrai).

Q : Quel est le risque algorithmique majeur lié à l'omission de la mise à jour des variables de contrôle dans le corps d'une boucle 'Tant Que' ?

R : Une boucle infinie (infinisation), provoquant le blocage du programme par épuisement des ressources processeur.

Q : Pourquoi l'utilisation de l'opérateur d'égalité stricte '==' pour comparer deux variables réelles à virgule flottante (Float) est-elle considérée comme une faute éliminatoire ?

R : En raison des imprécisions inhérentes à la représentation binaire des nombres à virgule flottante en mémoire, deux calculs mathématiquement égaux peuvent différer légèrement au niveau des derniers bits.